

UNITY GAME DEVELOPER PRACTICAL TEST

Project: **TopDown Tank Battleground** — Unity Upgrade, Cinemachine, Gamepad & Performance

CANDIDATE PRACTICAL TEST
TIME LIMIT: 2 DAYS MAX

Target Role: Unity Game Developer	Allowed Time: 2 Days Max (Approx. 6 – 8 hours of work)
Starting Unity Version: Unity 2021.3.33f1 (Built-in RP)	Target Unity Version: Latest Unity LTS (Unity 6 or 2022.3 LTS)
Controls: Full Gamepad Support + Keyboard/Mouse Fallback	Deliverables: Source Repo + Playable Build ZIP + Optimization Notes

Candidate Mission Brief: You are provided with a prototype Unity project (*TopDownTank-main*). It includes basic tank driving, turret aiming, projectile firing, and a simple enemy in a flat sandbox scene. However, the project has outdated engine files, mouse-only controls, a rigid camera, zero level progression, and noticeable lag caused by unoptimized code. Your task over the next **2 days** is to update the Unity version, replace the camera with Cinemachine, add full gamepad support, create a few playable levels, code a debug panel to track FPS, fix key performance bottlenecks, and output a playable standalone build.

1. Existing Codebase Review & Known Issues

Before writing new code, review the key flaws already identified in the project files that you need to resolve:

Area	Current File & Location	Observed Issue & What to Fix	Priority
Unity Version	ProjectSettings/ ProjectVersion.txt	Project is on older Unity 2021.3.33f1 . Needs to be updated to the latest Unity LTS build (Unity 6 or 2022.3 LTS) with clean package setup.	HIGH
Camera Script	Assets/Scripts/ TopDownCamera.cs	Script name mismatch (class is TopDownCameraFollow). Rigid follow with no screen shake, framing deadzones, or arena boundaries.	MEDIUM
Input Controls	Tank_Inputs.cs:50 ThrowProjectile.cs:25	Aiming is locked to Input.mousePosition and firing to Input.GetMouseButtonDown(0). Controllers/Gamepads do not work.	CRITICAL
Console Spam	Assets/Scripts/ EnemyTankAi.cs:160	Calls Debug.Log(distance); in FixedUpdate() every physics tick. Causes console flood and garbage collection (GC) micro-stutter.	CRITICAL
Bullet Spawning	Projectile.cs:11,22 ThrowProjectile.cs:41	Every bullet and particle effect is constantly spawned with Instantiate() and removed with Destroy(). No object pooling.	HIGH
Physics & Tagging	Projectile.cs:13,18 TankController.cs:41	Uses string comparison tag == 'PlayerTank' instead of CompareTag(). FixedUpdate() uses Time.deltaTime instead of Time.fixedDeltaTime.	MEDIUM
Levels & Flow	Assets/Scenes/ Test.unity	Only one flat test scene exists. No level progression, objective conditions, victory/loss screens, or menu.	MEDIUM

2. Candidate Tasks & Requirements

Below are the primary gameplay tasks to complete. Focus on clean code, good control feel, and solid execution within the 2-day timeframe.

TASK 1: Update Game Version to Latest Unity LTS Build

ENGINE UPGRADE

Goal: Upgrade the project from Unity 2021.3.33f1 to the latest official Unity LTS build (Unity 6 or 2022.3 LTS).

- Ensure all packages in Packages/manifest.json resolve cleanly.
- Fix any obsolete Unity APIs or deprecated calls so the console shows **zero compiler errors and zero warnings**.
- Briefly note any package changes in your submission README.md.

TASK 2: Use Cinemachine to Update the Camera

CAMERA & GAME FEEL

Goal: Remove the old TopDownCamera.cs script and implement a polished Unity Cinemachine camera.

- Install Cinemachine (com.unity.cinemachine) and create a Virtual Camera targeting the player tank.
- Configure smooth follow damping and an isometric top-down angle that gives clear sight of the battlefield.
- Add **Camera Shake** (Cinemachine Impulse) when the player fires the cannon and when projectiles explode.
- Add a **Cinemachine Confiner** or bounding area so the camera never shows empty grey space outside the arena.

TASK 3: Make the Game Fully Playable with a Gamepad

GAMEPAD & CONTROLS

Goal: Overhaul player controls so the game feels great on an Xbox or PlayStation controller, while keeping keyboard & mouse working.

• **Dual-Stick Controls:**

- *Left Stick:* Drives tank hull forward/backward and turns the chassis.
- *Right Stick:* Aims the turret smoothly in 360 degrees (with a reasonable deadzone so it doesn't drift).
- *Right Trigger (RT) or Bumper (RB):* Fires cannon shells at the normal fire-rate cadence.

• **Keyboard & Mouse Fallback:** WASD to drive, Mouse to aim reticle, Left Mouse Button to fire.

• **Seamless Switching:** Picking up the controller or touching the mouse should swap seamlessly without pausing or restarting.

• You may use Unity's New Input System or clean code on the legacy input manager.

TASK 4: Create Multiple Levels & Game Flow

LEVELS & PROGRESSION

Goal: Expand beyond the single test scene into a mini-campaign with at least **2 to 3 playable levels**.

• **Level Variety:**

- *Level 1 (Warmup):* Open arena, stationary or low-health target enemies, destructible blocks.
- *Level 2 (Tactical):* Barricades, narrow alleys, patrolling enemy tanks that hunt the player.
- *Level 3 (Showdown):* Multiple enemy tanks, defensive layout, higher difficulty.

• **Simple Game Loop:**

- Main Menu or Start Screen with a Play button.
- Win condition: Destroy all enemy tanks -> Victory screen with a 'Next Level' button.
- Lose condition: Player health reaches 0 -> Game Over screen with a 'Retry' button.

2. Candidate Tasks (Continued)

TASK 5: Code an In-Game Debug Panel (FPS Counter & QA Tools)

DEBUGGING TOOLS

Goal: Build a toggleable UI debug overlay so testers can monitor framerate and test features quickly.

- **Toggle Hotkey:** Easily opened/closed using a key (e.g. F3 / Backquote) and a controller shortcut (e.g. Select/View button).
- **Performance Metrics Displayed:**
 - **Live FPS:** Show Current FPS, Smoothed Average FPS, and 1% Low / Min FPS.
 - **Frame Time:** Frame delta in milliseconds (e.g. 16.6 ms).
 - **Memory Usage:** Current allocated heap memory.
 - **Entity Counter:** Count of active enemies and active bullets currently in the scene.
- **Quick QA Cheats:**
 - *God Mode:* Toggle player tank invincibility.
 - *Spawn Enemy:* Button to spawn an extra enemy tank for testing.
 - *Win / Skip Level:* Instantly triggers level completion to jump to the next stage.
 - *Game Speed (TimeScale):* Slider or buttons for Slow Motion (0.5x) and Fast (1.5x).
- **Efficiency:** Ensure the debug panel updates without generating garbage strings every frame.

TASK 6: Performance Optimization & Optimization Tips

OPTIMIZATION

Goal: Fix the existing lag sources in the codebase and write down clear optimization recommendations.

- **In-Code Fixes Required:**
 1. Delete the `Debug.Log(distance)`; call inside `EnemyTankAi.cs:160`.
 2. Implement a simple **Object Pool** for bullets and explosion particle effects instead of calling `Instantiate` and `Destroy` every shot.
 3. Replace string comparisons (`tag == 'PlayerTank'`) with `CompareTag()`.
 4. In `TankController.cs` and `EnemyTankAi.cs`, replace `Time.deltaTime` inside `FixedUpdate()` with `Time.fixedDeltaTime`.
 5. Add a `LayerMask` to the aiming raycast so it only hits the ground plane, not the tank itself.

- **Written Optimization Tips (OPTIMIZATION_TIPS.md):**

Provide a concise 1-page document proposing practical tips for how this game can run at locked high FPS (Draw calls, static batching, light baking, physics collision layer matrix, texture compression).

TASK 7: Make a Playable Standalone Build

FINAL BUILD

Goal: Export a working standalone Windows build so the game can be tested directly outside Unity.

- Target platform: **Windows Standalone (x86_64)**.
- Set product name to 'Tank Battleground' and configure default windowed/fullscreen options.
- Verify that both gamepad controls and keyboard/mouse work properly in the compiled build.
- Package the executable and data files into a single ZIP file for submission.

3. Scoring Rubric & Submission Checklist

Submissions are evaluated on a 100-point rubric. We look for good control feel, clean readable code, and working features.

Category	Points	Evaluation Criteria
1. Unity LTS Upgrade	15 pts	Project opens in the target Unity LTS with zero errors and zero warnings. Packages resolve cleanly.
2. Cinemachine Camera	15 pts	Smooth top-down follow, camera shake on firing/explosions, and confiner bounds preventing view clipping.
3. Gamepad & Controls	20 pts	Dual-stick drive & 360° aim feels natural on controller with deadzones. Triggers fire. KBM fallback works.
4. Levels & Game Loop	15 pts	2 to 3 playable levels with obstacle variety, enemy tanks, win/loss UI, and smooth scene transitions.
5. FPS Debug Panel	15 pts	Working in-game overlay showing live FPS, frame ms, memory, and functional cheats (God Mode, Spawner, Skip).
6. Optimization & Build	20 pts	Log spam removed, bullet pooling working, tag check fixed, written optimization tips, and playable Windows build.
Total Score	100 pts	Passing score: 75+ points. (Bonus: +5 points for extra audio polish, particle effects, or controller rumble).

Submission Procedure & Checklist (2-Day Window)

Deliverable	Details	Status
1. Git Repository	GitHub/GitLab repository link (exclude Library/, Temp/, and Logs/).	[] Done
2. Standalone Build	Playable Windows 64-bit build (.exe) zipped and shared via Google Drive or GitHub release.	[] Done
3. Optimization Tips	A short OPTIMIZATION_TIPS.md explaining the fixes made and future optimization ideas.	[] Done
4. Readme Notes	Quick README.md explaining control mappings, Unity version used, and debug panel hotkey.	[] Done

Practical FAQ for Candidates

Q: Can I use Unity's New Input System?

A: Yes. You may use either Unity's New Input System package or write clean custom input code using the Input Manager. Whichever you choose, ensure the gamepad dual-stick controls feel smooth and responsive.

Q: What if I run out of time on 3 levels?

A: 2 well-designed levels with working win/loss flow and good combat feel is better than 3 rushed broken scenes. Focus on quality and smooth gameplay feel first.

Q: Can I use free sound effects or particle assets?

A: Yes, you are welcome to use free assets to improve game feel, but all game logic (gamepad input, camera shake, pooling, debug panel) should be written by you.